



Introduzione ad Agda: programmazione funzionale con tipizzazione dipendente

Università di Verona
Imbriani Paolo - VR500437

12 novembre 2025

Indice

1	Introduzione	4
1.1	Che cos'è la Teoria dei Tipi?	4
1.2	Cos'è Agda?	4
1.2.1	Caratteristiche principali di Agda	5
2	Primi passi con Agda	5
2.1	Tipi di base	5
2.2	Definizione di operatori	6
2.3	Pattern matching e Goals	6
2.4	Liste	7
2.5	Maybe	8
3	Uguaglianza in Agda	9
3.1	Definizione di uguaglianza	9
3.2	Congruenza	11
3.3	Simmetria	12
3.4	Transitività	12
3.5	Associatività	12
3.6	Commutatività	13
4	Logica con PaT (Propositions as Types)	14
4.1	Connettivi logici in Agda	14
4.1.1	Proprietà interessanti da dimostrare	16
4.2	Leggi classiche della logica	18
4.3	Logica del primo ordine	20
4.3.1	Assioma della scelta	22
5	Metalogica in Agda	23
5.1	Definizione delle proposizioni e formule	23
5.2	Definizione dei contesti	23
5.3	Definizione delle regole di deduzione naturale	24
5.4	Esempio di prova in Agda	25
5.5	Semantica delle formule	27
5.6	Teorema di correttezza	28
5.7	Teorema di completezza (e perché non è sempre realizzabile in Agda)	30

Prefazione

Questa dispensa e raccolta di appunti è stata realizzata durante il tirocinio del terzo anno di Informatica (A.A 2025/2026) presso l'Università di Verona. L'obiettivo principale di questo lavoro è quello di aiutarmi nel corso del tirocinio per avere le conoscenze di base di Agda per poter modellare la logica modale S4.2 utilizzando come strumento proprio Agda. All'interno di questo documento vengono usati anche estratti di lezioni del mini-corso «*Introduction to Type Theory*» tenute dal professore *Thorsten Altenkirch*, docente di Informatica presso l'Università di Nottingham. Questa dispensa non vuole **sostituire** i testi ufficiali sulla teoria dei tipi o su Agda, ma vuole essere un **supporto** per chi si avvicina per la prima volta a questo linguaggio di programmazione funzionale con tipizzazione dipendente.

1 Introduzione

1.1 Che cos'è la Teoria dei Tipi?

Originariamente sviluppata come alternativa alla teoria degli insiemi (*Axiomatic Set Theory*) per fondare la matematica, la teoria dei tipi è ora ampiamente utilizzata in informatica, in particolare nei linguaggi di programmazione funzionale e nei sistemi di verifica formale. Non solo, molto semplicemente la teoria dei tipi la possiamo vedere come una logica costruttiva ed intuizionista.

Caratteristiche principali

- **Il typing come giudizio:**

$$3 \in \mathbb{N} \quad \text{o equivalentemente} \quad 3 : \mathbb{N}$$

Nella Teoria dei Tipi, il typing è un giudizio, non una proposizione. La Teoria dei Tipi è staticamente tipata — la correttezza dei tipi è verificata dal sistema stesso.

- **Tipi dipendenti:** I tipi possono dipendere dai valori. Prendiamo per esempio il tipo dei vettori:

$$\text{Vec} : \mathbb{N} \rightarrow \text{Type} \quad \text{dove} \quad \text{Vec}(n) \text{ è il tipo dei vettori di lunghezza } n.$$

- **Assioma di univalenza:** Identifica l'equivalenza di strutture con l'uguaglianza di tipi (come nella Homotopy Type Theory).
- **Operazioni insiemistiche:** Le usuali operazioni $\cup, \cap, \subseteq$ non sono definite direttamente sui tipi, ma sui sottoinsiemi (o predicati) di un dato tipo.
- **La logica è emergente:** La corrispondenza tra logica e tipi è nota come isomorfismo di Curry–Howard:

Proposizioni come Tipi, Dimostrazioni come Programmi.

- **Le funzioni sono primitive:** Le funzioni sono oggetti centrali nella Teoria dei Tipi. Ad esempio, una relazione R su tipi A, B può essere vista come:

$$R : A \rightarrow B \rightarrow \text{Prop}$$

1.2 Cos'è Agda?

Agda è un linguaggio di programmazione funzionale con tipizzazione dipendente e un sistema di dimostrazione interattivo. Sviluppato principalmente per la ricerca e l'insegnamento, Agda consente di scrivere programmi che sono anche prove formali, garantendo così la correttezza del codice attraverso il sistema di tipi.

1.2.1 Caratteristiche principali di Agda

- **Sintassi simile a Haskell:** Agda ha una sintassi che ricorda quella di Haskell, rendendolo accessibile a chi ha familiarità con i linguaggi funzionali.
- **Supporto Unicode:** Agda supporta caratteri Unicode, permettendo l'uso di simboli matematici direttamente nel codice.
- **Tipi dipendenti:** Agda supporta tipi che possono dipendere da valori, permettendo di esprimere proprietà più precise nei tipi stessi.
- **Propositions as Types (PaT):** In Agda, le proposizioni logiche sono rappresentate come tipi, e le prove di queste proposizioni sono rappresentate come programmi che abitano questi tipi.
- **Sistema di moduli:** Agda ha un sistema di moduli che consente di organizzare il codice in modo strutturato e riutilizzabile.
- **Interattività:** Agda offre un ambiente interattivo che facilita la scrittura e la verifica delle prove, con feedback immediato sullo stato delle dimostrazioni.

Agda ci permette di modellare concetti matematici e logici in modo rigoroso, fornendo un ponte tra teoria dei tipi, programmazione funzionale e logica formale. La cosa interessante è che funziona come se stessimo costruendo le fondamenta su cui poi poggeranno i nostri assiomi o dimostrazioni (è proprio così che funziona la logica costruttiva).

💡 Osservazione 1.1

Agda è un linguaggio di programmazione funzionale totale, il che significa che ogni funzione definita in Agda deve essere totale, ovvero deve essere definita per tutti gli input possibili e deve sempre terminare. Senza questa caratteristica, la logica alla base del linguaggio diventerebbe inconsistente, e sarebbe possibile dimostrare affermazioni arbitrarie.

2 Primi passi con Agda

2.1 Tipi di base

In agda, come già precedentemente detto, niente è realmente definito fin dall'inizio (se non dalle librerie standard di Agda). Tuttavia possiamo noi stessi definire i nostri tipi di base. Per esempio, definiamo il tipo dei numeri naturali, usando la notazione di Peano:

```
data ℕ : Set where
  zero : ℕ
  suc  : ℕ → ℕ
```

In questo tipo di dato, abbiamo due costruttori:

- zero rappresenta il numero naturale 0.
- suc è una funzione che prende un numero naturale n e restituisce il successore di n , cioè $n + 1$.

🔔 Osservazione 2.1

In Agda, tutto può essere un nome anche la stringa `134function_*3` poiché l'unico vincolo che abbiamo sui nomi è che non possono contenere spazi (poiché Agda usa gli spazi per definire i nomi).

2.2 Definizione di operatori

Possiamo definire operatori personalizzati in Agda per rendere il codice più leggibile. Ad esempio, definiamo l'operatore di somma per i numeri naturali:

```
_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)
```

In questo esempio, l'operatore `+` è definito come una funzione che prende due numeri naturali e restituisce la loro somma. Altro esempio, definiamo l'operatore di moltiplicazione:

```
_*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)
```

Solitamente, in Agda, per aggiungere chiarezza a questi operatori dobbiamo anche specificare la loro precedenza e associatività attraverso le direttive `infix` e `infixl/infixr`.

- `infixl n _op_`: Definisce l'operatore `op` come un operatore binario con associatività a sinistra e precedenza n .
- `infixr n _op_`: Definisce l'operatore `op` come un operatore binario con associatività a destra e precedenza n .
- `infix n _op_`: Definisce l'operatore `op` come un operatore binario non associativo con precedenza n .

Il valore n determina la precedenza dell'operatore, con valori più alti che indicano una precedenza più alta. Solitamente si usa una scala da 0 a 9 per la precedenza degli operatori.

2.3 Pattern matching e Goals

Agda aiuta nella scrittura delle funzioni tramite il **pattern matching**. Quando definiamo una funzione, possiamo specificare come essa si comporta per diversi casi basati sui valori di input.

Prendiamo come esempio la funzione somma che abbiamo scritto sopra, un modo per farci aiutare da Agda è quello di scrivere la firma della funzione e poi usare il comando **C-c C-l** per creare i **goals** (obiettivi) che dobbiamo soddisfare per completare la definizione della funzione.

```

_+_ : ℕ → ℕ → ℕ
_+_ = ?

```

Dopo aver premuto **C-c C-l**, Agda genererà i seguenti goals:

```

_+_ : ℕ → ℕ → ℕ
_+_ = {!0!}

```

Dove `{!0!}` rappresenta un buco che dobbiamo riempire per completare la definizione della funzione. Possiamo quindi completare la definizione della funzione somma utilizzando il pattern matching per specificare i casi. Basta inserire prima del simbolo `=` i parametri `m` e `n` e poi premere **C-c C-c** per generare i goals per ciascun caso:

```

_+_ : ℕ → ℕ → ℕ
zero + n = {!0!}
suc m + n = {!1!}

```

Ora, dobbiamo riempire i buchi con le espressioni appropriate per ciascun caso.

```

_+_ : ℕ → ℕ → ℕ
zero + n = {! n !}
suc m + n = {! suc (m + n) !}

```

Dopo aver completato la definizione, possiamo rimuovere i buchi facendo la combinazione di tasti **C-c C-SPACE** per verificare che la funzione sia correttamente definita. Alternativamente alla ridefinizione di ogni singolo concetto base, possiamo importare le librerie standard di Agda che contengono già definizioni di tipi e funzioni comuni.

2.4 Liste

Definiamo il tipo delle liste in Agda:

```

data List (A : Set) : Set where
  [] : List A
  _::_ : A → List A → List A

```

In questa definizione:

- `List` è un tipo parametrizzato da un tipo `A`, che rappresenta il tipo degli elementi della lista.

- `[]` rappresenta la lista vuota.
- `_ :: _` è un costruttore che prende un elemento di tipo A e una lista di tipo $List\ A$, restituendo una nuova lista con l'elemento aggiunto all'inizio.

Proviamo a definire la funzione `length` (per calcolare la lunghezza di una lista):

```
length : {A : Set} → List A → ℕ
length [] = zero
length (x :: xs) = suc (length xs)
```

In questa definizione:

- Prima di tutto, notiamo che la funzione `length` è parametrizzata da un tipo A (usando le parentesi graffe per indicare che è un parametro implicito e quindi non deve essere specificato esplicitamente quando si chiama la funzione).
- La funzione `length` prende una lista di tipo $List\ A$ e restituisce un numero naturale $\mathbb{N}$.
- Nel caso della lista vuota, la lunghezza è zero.
- Nel caso di una lista non vuota, la lunghezza è il successore della lunghezza della coda della lista.

Notiamo come il modo in cui definiamo le funzioni in Agda è in maniera ricorsiva e tramite pattern matching, che ci permette di definire il comportamento della funzione per diversi casi basati sui valori di input. Abbiamo un caso base (per la lista vuota) e un caso ricorsivo (per la lista non vuota).

2.5 *Maybe*

Per definire l'operatore di estrazione di un elemento da una lista, possiamo utilizzare il tipo `Maybe` per gestire il caso in cui l'elemento non esista.

```
data Maybe (A : Set) : Set where
  nothing : Maybe A
  just : A → Maybe A
```

In questa definizione:

- `Maybe` è un tipo parametrizzato da un tipo A , che rappresenta il tipo dell'elemento opzionale.
- `nothing` rappresenta l'assenza di un valore.
- `just` è un costruttore che prende un elemento di tipo A e restituisce un valore di tipo `Maybe A` che contiene quell'elemento.

Ora possiamo definire la funzione `_ !! _` per estrarre l' n -esimo elemento da una lista:

```

_!!_ : {A : Set} → List A → ℕ → Maybe A
[] !! n = nothing
(x :: xs) !! zero = just x
(x :: xs) !! (suc n) = xs !! n

```

In quest'altra definizione notiamo:

- La funzione `_!!_` è parametrizzata da un tipo A (usando le parentesi graffe per indicare che è un parametro implicito).
- La funzione prende una lista di tipo `List A` e un numero naturale $\mathbb{N}$ (l'indice dell'elemento da estrarre) e restituisce un valore di tipo `Maybe A`.
- Nel caso della lista vuota, restituisce `nothing`, indicando che non c'è nessun elemento da estrarre.
- Nel caso in cui l'indice sia zero, restituisce il primo elemento della lista avvolto in `just`.
- Nel caso in cui l'indice sia un successore, chiama ricorsivamente la funzione sulla coda della lista e sull'indice decrementato di uno.

Questo costrutto ci permette di gestire in modo sicuro l'estrazione di elementi da una lista, senza incorrere in errori di accesso a indici fuori dai limiti. Questa caratteristica è molto più comune in Haskell che in Agda, ma penso sia comunque interessante mostrarla.

🔔 Osservazione 2.2

Tuttavia, esiste una versione più sicura di questa funzione che sfrutta i tipi dipendenti che garantisce che l'indice sia sempre valido per ogni input, evitando di usare il tipo `Maybe`. È interessante notare come in Agda, con il giusto impegno e l'uso delle dimostrazioni formali, possiamo garantire la correttezza del nostro codice facendo in modo che *non cada mai in errori di runtime*.

3 Uguaglianza in Agda

In Agda, l'uguaglianza è un concetto fondamentale che viene trattato in modo rigoroso attraverso il sistema di tipi.

3.1 Definizione di uguaglianza

L'uguaglianza in Agda è definita tramite un tipo chiamato `_≡_`, che rappresenta l'uguaglianza tra due elementi di uno stesso tipo.

```
data _≡_ {A : Set} (x : A) : A → Set where
  refl : x ≡ x
```

In questa definizione:

- `_≡_` è un tipo parametrizzato da un tipo A e da un elemento x di tipo A .
- Il costruttore `refl` rappresenta il fatto che ogni elemento è uguale a se stesso.

Da qua, inizia tutto il lavoro di dimostrazione delle proprietà dell'uguaglianza. Infatti «`refl`» (che è nient'altro che la riflessività) è il blocchetto principale per poi costruire tutte le altre definizioni.

💡 Osservazione 3.1

Nella matematica tradizionale " $x = y$ " è un'affermazione, una dichiarazione. Ad esempio, questa affermazione potrebbe essere vera o falsa. In Agda invece, " $x \equiv y$ " è un tipo: il tipo dei testimoni che dimostrano che x e y sono uguali. Ad esempio, questo tipo potrebbe essere abitato (cioè esistere un valore di quel tipo), oppure potrebbe essere vuoto (cioè non esistere alcun valore di quel tipo). Quindi se nella matematica tradizionale dimostreremmo che $x = y$, in Agda esibiamo un valore di tipo $x \equiv y$.

Con la definizione di uguaglianza a disposizione, possiamo affermare e dimostrare che zero è uguale a zero:

```
zero-equals-zero : zero ≡ zero
zero-equals-zero = refl
```

In esattamente lo stesso modo, possiamo affermare e dimostrare che $2 + 2 = 4$:

```
grande-teorema : two + two ≡ four
grande-teorema = refl
```

Possiamo usare `refl` in esattamente quelle situazioni, in cui i due lati della presunta identità sono manifestamente uguali, cioè uguali in un modo che richiede solo lo srotolamento di tutte le definizioni e la semplificazione tramite il calcolo, ma nessuna vera intuizione, per convincersi dell'uguaglianza. Per esempio, essendo che $\text{zero} + b$ è definito come b , possiamo usare `refl` nella seguente dimostrazione:

```
trivial : (b : ℕ) → zero + b ≡ b
trivial b = refl
```

Questo tuttavia funziona solo perché abbiamo definito l'operatore di somma in un certo modo.

```

_+_ : ℕ → ℕ → ℕ
zero + n = n -- Guarda qui!
suc m + n = suc (m + n)

```

È tutt'altra storia il verso opposto dell'uguaglianza, cioè dimostrare che b è uguale a $zero + b$. In questo caso, non possiamo usare `refl`, perché b non è definito come $zero + b$. Dobbiamo prima però definire un paio di strumenti ausiliari.

3.2 Congruenza

La **congruenza** è una proprietà fondamentale dell'uguaglianza che afferma che se due elementi sono uguali, allora lo sono anche i risultati delle funzioni applicate a questi elementi. Proviamo a definire la funzione di congruenza in Agda:

```

cong : {A B : Set} {x y : A}
→ (f : A → B)
→ x ≡ y
→ f x ≡ f y
cong f refl = refl

```

In questa definizione:

- La funzione `cong` prende due tipi A e B , due elementi x e y di tipo A , una funzione f da A a B , e una prova che x è uguale a y .
- Restituisce una prova che $f(x)$ è uguale a $f(y)$.
- Il caso base della definizione afferma che se la prova di uguaglianza è data da `refl`, allora anche l'uguaglianza tra $f(x)$ e $f(y)$ è dimostrata da `refl`.

Ora possiamo effettivamente dimostrare che b è uguale a $zero + b$ usando la funzione di congruenza:

```

symm-zero-plus : (b : ℕ) → b + zero ≡ b
symm-zero-plus zero = refl
symm-zero-plus (suc n) = cong suc (symm-zero-plus n)

```

Vediamo come:

- Nel caso base, quando b è zero, la prova è semplicemente `refl`, poiché zero è uguale a $zero + zero$. (Stiamo praticamente usando la riflessività per dire che zero è uguale a zero).
- Nel caso ricorsivo, quando b è il successore di n , usiamo la funzione di congruenza per applicare la funzione `suc` alla prova che n è uguale a $zero + n$.

3.3 Simmetria

La **simmetria** è un'altra proprietà fondamentale dell'uguaglianza che afferma che se un elemento è uguale a un secondo, allora il secondo è uguale al primo. Proviamo a definire la funzione di simmetria in Agda:

```
sym : {A : Set} → {x y : A}
  → x ≡ y
  → y ≡ x
sym refl = refl
```

3.4 Transitività

La **transitività** è un'altra proprietà fondamentale dell'uguaglianza che afferma che se un elemento è uguale a un secondo, e il secondo è uguale a un terzo, allora il primo è uguale al terzo. Proviamo a definire la funzione di transitività in Agda:

```
trans : {A : Set} → {x y z : A}
  → x ≡ y
  → y ≡ z
  → x ≡ z
trans refl refl = refl
```

Ormai abbiamo capito che se la prova di uguaglianza è data da `refl`, allora possiamo concludere che l'uguaglianza tra x e z è dimostrata da `refl`.

3.5 Associatività

Possiamo anche dimostrare che l'operazione di somma è associativa utilizzando le proprietà dell'uguaglianza che abbiamo definito.

```
assoc-+ : (a b c : ℕ) → (a + b) + c ≡ a + (b + c)
assoc-+ zero b c = refl
assoc-+ (suc a) b c = cong suc (assoc-+ a b c)
```

In questa definizione:

- Nel caso base, quando a è zero, la prova è semplicemente `refl`, poiché zero sommato a qualsiasi numero è uguale a quel numero.
- Nel caso ricorsivo, quando a è il successore di a' , usiamo la funzione di congruenza per applicare la funzione `suc` alla prova che $a' + b + c$ è uguale a $a' + (b + c)$.

3.6 Commutatività

Possiamo anche dimostrare che l'operazione di somma è commutativa utilizzando le proprietà dell'uguaglianza che abbiamo definito.

```

comm-+ : (a b : ℕ) → a + b ≡ b + a
comm-+ zero b = symm-zero-plus b
comm-+ (suc a) b = cong suc (comm-+ a b)

```

💡 Osservazione 3.2

Come notiamo man mano che costruiamo le definizioni di queste proprietà, esse ci aiutano a costruire dimostrazioni più complesse basate su di esse (perché sono "witness" o "testimoni" che ci permettono di affermare certe uguaglianze). La definizione di uguaglianza in Agda e le proprietà che ne derivano (riflessività, simmetria, transitività, congruenza) forniscono una base solida per ragionare sull'uguaglianza tra elementi di tipi diversi. Questi concetti sono fondamentali non solo in Agda, ma anche in molte altre aree della matematica e dell'informatica teorica.

In questa definizione:

- Come parametro, abbiamo due numeri naturali a e b . L'obiettivo è ottenere una dimostrazione che dice che $a + b$ è uguale a $b + a$.
- Nel caso base, quando a è zero, usiamo la funzione `symm-zero-plus` per dimostrare che $b + 0$ è uguale a b .
- Nel caso ricorsivo, quando a è il successore di a' , usiamo la funzione di congruenza per applicare la funzione `suc` alla prova che $a' + b$ è uguale a $b + a'$.

4 Logica con PaT (Propositions as Types)

4.1 Connettivi logici in Agda

In Agda, la logica è trattata attraverso il paradigma delle **Proposizioni come Tipi** (PaT, Propositions as Types). Questo paradigma stabilisce una corrispondenza tra proposizioni logiche e tipi di dati, dove una proposizione è vera se e solo se esiste un termine (un valore) del tipo corrispondente. Possiamo definire l'insieme di proposizioni in Agda in questa maniera:

```
Prop = Set

variable P Q R : Prop

infixr 1 _⇒_
_⇒_ : Prop → Prop → Prop
P ⇒ Q = P → Q
```

Nella definizione qua sopra abbiamo descritto anche l'implicazione logica $P \Rightarrow Q$ attraverso una funzione che prende un termine di tipo P e restituisce un termine di tipo Q . Questa corrispondenza tra funzioni e connettivi logici ci permette di definire altri connettivi logici in modo simile. Possiamo provare questo tipo di proposizione:

$$(P \rightarrow Q \rightarrow R) \rightarrow (Q \rightarrow P \rightarrow R)$$

In questa maniera:

```
swap : (P ⇒ Q ⇒ R) ⇒ (Q ⇒ P ⇒ R)
swap pqr q p = pqr p q
```

Banalmente stiamo definendo una funzione `swap` che prende come input una funzione di tipo $P \Rightarrow Q \Rightarrow R$ e restituisce una funzione di tipo $Q \Rightarrow P \Rightarrow R$.

- La funzione `swap` prende come argomento una funzione `pqr` di tipo $P \Rightarrow Q \Rightarrow R$.
- Per il concetto di currying possiamo anche prendere due argomenti separati, q di tipo Q e p di tipo P .
- A questo punto possiamo applicare la funzione `pqr` agli argomenti p e q nell'ordine "scambiato" per ottenere un termine di tipo R .

Andiamo avanti a definire la congiunzione logica $P \wedge Q$ come un tipo di coppia. Però prima di tutto, ci tocca definire il tipo delle coppie in Agda:

```

record _×_ (A B : Set) : Set where
  constructor _,_
  field
    proj1 : A
    proj2 : B

open _×_

```

Capiamo bene questa definizione:

- La parola chiave `record` viene usata per definire un tipo di dato che rappresenta una struttura composta da più campi.
- Il costruttore `_,_` viene usato per creare istanze del tipo di dato coppia.
- I campi `proj1` e `proj2` rappresentano i due elementi della coppia, rispettivamente di tipo A e B . Per semplicità potevamo anche chiamarli `fst` e `snd` per `first` e `second`.
- Quindi possiamo accedere ai campi di una coppia usando `proj1` e `proj2`.
- Inoltre l'istruzione `open _×_` ci permette di accedere ai campi del record senza dover specificare il nome del record ogni volta.

Con questa definizione a disposizione, possiamo ora definire la congiunzione logica $P \wedge Q$:

```

infix 3 _∧_
_∧_ : Prop → Prop → Prop
P ∧ Q = P × Q

```

Qui abbiamo definito l'operatore $\wedge$ come un operatore binario che prende due proposizioni P e Q . Ora che abbiamo l'implicazione e la congiunzione, possiamo facilmente definire il "se e solo se" $P \Leftrightarrow Q$ come segue:

```

infix 0 _⇔_
_⇔_ : Prop → Prop → Prop
P ⇔ Q = (P ⇒ Q) ∧ (Q ⇒ P)

```

Poiché $P \Leftrightarrow Q$ è vero se e solo se sia $P \Rightarrow Q$ che $Q \Rightarrow P$ sono veri. Ora, pensiamo ora a come fare la disgiunzione. In Agda, possiamo definire la disgiunzione logica $P \vee Q$ come un tipo somma (o tipo di unione). Questo è fatto tramite u-plus, ovvero un operatore insiemistico per l'unione disgiunta. Ecco come possiamo definirlo:

```

data _⊔_ (A B : Set) : Set where
  inj1 : A → A ⊔ B
  inj2 : B → A ⊔ B

```

Dobbiamo definire l'unione disgiunta come data type perché ha due costruttori distinti:

- inj_1 prende un elemento di tipo A e lo inserisce nell'unione disgiunta $A \uplus B$.
- inj_2 prende un elemento di tipo B e lo inserisce nell'unione disgiunta $A \uplus B$.

Ora che abbiamo questo strumento, possiamo definire la disgiunzione logica $P \vee Q$:

```
infix 2 _V_  
_V_ : Prop → Prop → Prop  
P V Q = P  $\uplus$  Q
```

Ci mancano solo la negazione e la falsità per completare i connettivi logici di base.

```
data  $\perp$  : Set where  
  
record  $\top$  : Set where  
  
 $\neg$  : Prop → Prop  
 $\neg$  P = P  $\Rightarrow$   $\perp$ 
```

In queste definizioni importanti abbiamo:

- La falsità $\perp$ è definita come un tipo di dato vuoto, che non ha costruttori.
- La verità $\top$ è definita come un record vuoto, che ha esattamente un costruttore (quindi è sempre abitato).
- La negazione $\neg P$ è definita come l'implicazione da P a $\perp$, il che significa che $\neg P$ è vero se e solo se P porta a una contraddizione (cioè alla falsità).

4.1.1 Proprietà interessanti da dimostrare

Una proprietà interessante da dimostrare è quella dell'associazione della congiunzione. Per questa prova abbiamo bisogno di dimostrare sia un lato che l'altro:

```
assoc : P  $\wedge$  (Q  $\wedge$  R)  $\Leftrightarrow$  (P  $\wedge$  Q)  $\wedge$  R  
proj1 assoc (p , (q , r)) = (p , q) , r  
proj2 assoc ((p , q) , r) = p , (q , r)
```

Cerchiamo di capire questa definizione:

- La funzione `assoc` prende come argomento una coppia di proposizioni $P \wedge (Q \wedge R)$ e restituisce una coppia di proposizioni $(P \wedge Q) \wedge R$.
- Per dimostrare la prima direzione dell'equivalenza, usiamo il campo `proj1` per estrarre gli elementi dalla coppia $P \wedge (Q \wedge R)$ e li riorganizziamo per formare la coppia $(P \wedge Q) \wedge R$.

- Per dimostrare la seconda direzione dell'equivalenza, usiamo il campo `proj2` per estrarre gli elementi dalla coppia $(P \wedge Q) \wedge R$ e li riorganizziamo per formare la coppia $P \wedge (Q \wedge R)$.
- Abbiamo usato il costruttore delle coppie $(\ , \)$ per creare nuove coppie di proposizioni.

🔔 Osservazione 4.1

Ricordiamoci di stare attenti allo spazio tra termine e operatori in Agda.

Proviamo ora a dimostrare la distributività della congiunzione sulla disgiunzione, ovvero:

$$P \wedge (Q \vee R) \Leftrightarrow (P \wedge Q) \vee (P \wedge R)$$

Questa prova è leggermente più complicata quindi cerchiamo di andare a step:

```

distrib : P ∧ (Q ∨ R) ⇔ (P ∧ Q) ∨ (P ∧ R)
proj1 distrib (p , inj1 q) = inj1 (p , q)
proj1 distrib (p , inj2 r) = inj2 (p , r)
proj2 distrib (inj1 (p , q)) = p , inj1 q
proj2 distrib (inj2 (p , r)) = p , inj2 r

```

Come possiamo notare abbiamo anche qua diviso la dimostrazione in due parti a causa del "se e solo se":

- Ora, la funzione `distrib` prende come argomento una coppia di proposizioni $P \wedge (Q \vee R)$ e restituisce una disgiunzione di coppie di proposizioni $(P \wedge Q) \vee (P \wedge R)$.
- Vediamo come la direzione "a destra" viene dimostrata usando il campo `proj1`:
 - Se la seconda proposizione della coppia è costruita usando `inj1`, allora estraiamo q e formiamo la coppia $(P \wedge Q)$ usando p e q , che viene poi inserita nella disgiunzione usando `inj1`.
 - Se la seconda proposizione della coppia è costruita usando `inj2`, allora estraiamo r e formiamo la coppia $(P \wedge R)$ usando p e r , che viene poi inserita nella disgiunzione usando `inj2`.
- La direzione "a sinistra" invece, viene dimostrata usando il campo `proj2`:
 - Se la disgiunzione è costruita usando `inj1`, allora estraiamo la coppia (p, q) e formiamo la coppia $P \wedge (Q \vee R)$ usando p e `inj1 q`.
 - Se la disgiunzione è costruita usando `inj2`, allora estraiamo la coppia (p, r) e formiamo la coppia $P \wedge (Q \vee R)$ usando p e `inj2 r`.

4.2 Leggi classiche della logica

E l'ex falso quodlibet? Possiamo dimostrare l'ex falso quodlibet, che afferma che da una contraddizione (falsità) possiamo dedurre qualsiasi proposizione.

```
ex-falso :  $\perp \Rightarrow P$ 
ex-falso ()
```

Questa definizione è particolare perché:

- La funzione `ex-falso` prende come argomento una prova di falsità $\perp$ e restituisce una prova di qualsiasi proposizione P .
- Il corpo della funzione utilizza la sintassi di pattern matching con una parentesi vuota `()`, che indica che non ci sono casi da considerare, poiché non esistono prove di falsità.
- Quindi, questa definizione afferma che se abbiamo una prova di falsità, possiamo dedurre qualsiasi proposizione P , il che è coerente con il principio logico dell'ex falso quodlibet.

Un altro principio logico interessante è il principio del terzo escluso, che afferma che per ogni proposizione P , o P è vera o la sua negazione $\neg P$ è vera. In Agda, possiamo definire questo principio come segue:

```
tnd : Prop  $\rightarrow$  Prop
tnd P = (P  $\vee$   $\neg$  P)
```

Inoltre possiamo anche provare la reductio ad absurdum (riduzione all'assurdo):

```
raa : Prop  $\rightarrow$  Prop
raa P = ( $\neg$  ( $\neg$  P))  $\Rightarrow$  P
```

Infine, possiamo dimostrare che il principio del terzo escluso implica la reductio ad absurdum:

```
tnd $\rightarrow$ raa : (tnd P)  $\Rightarrow$  raa P
tnd $\rightarrow$ raa (inj1 p) nnp = p
tnd $\rightarrow$ raa (inj2 np) nnp = efq (nnp np)
```

Guardiamo come abbiamo costruito questa definizione:

- La funzione `tnd $\rightarrow$ raa` prende come argomento una prova del principio del terzo escluso `tnd P` e restituisce una prova della reductio ad absurdum `raa P`.
- Nel corpo della funzione, usiamo il pattern matching per distinguere i due casi del principio del terzo escluso:
 - Se la prova è costruita usando `inj1`, allora abbiamo una prova diretta di P , che possiamo restituire direttamente.

- Se la prova è costruita usando inj_2 , allora abbiamo una prova della negazione di P (cioè $\neg P$). In questo caso, usiamo la funzione efq (ex falso quodlibet) per dedurre P dalla contraddizione ottenuta applicando la negazione di P alla prova della negazione di P .

Notiamo come possiamo dimostrare la "non-falsità" del principio del terzo escluso:

```
nntnd : ¬ (¬ (tnd P))
nntnd f = f (inj₂ (λ p → f (inj₁ p)))
```

Stiamo attenti a questa definizione:

- La funzione `nntnd` prende come argomento una prova della negazione del principio del terzo escluso $\neg(\text{tnd } P)$ e restituisce una prova della non-falsità del principio del terzo escluso $\neg\neg(\text{tnd } P)$.
- Nel corpo della funzione, applichiamo la prova della negazione del principio del terzo escluso f a una disgiunzione costruita usando inj_2 .
- La disgiunzione è costruita come una funzione anonima $\lambda p \rightarrow f (\text{inj}_1 p)$, che prende una prova di P e la usa per costruire una disgiunzione usando inj_1 .
- Quindi, questa definizione afferma che se abbiamo una prova della negazione del principio del terzo escluso, possiamo dedurre una contraddizione, il che implica che il principio del terzo escluso non può essere falso.

Notiamo come ora possiamo anche provare che la *raa* implica il principio del terzo escluso:

```
raa→tnd : raa (tnd P) → tnd P
raa→tnd h = h nntnd
```

Molto semplicemente:

- La funzione `raa→tnd` prende come argomento una prova della reductio ad absurdum del principio del terzo escluso $\text{raa } (\text{tnd } P)$ e restituisce una prova del principio del terzo escluso $\text{tnd } P$.
- Nel corpo della funzione, applichiamo la prova della reductio ad absurdum h alla prova della non-falsità del principio del terzo escluso `nntnd`.
- Quindi, questa definizione afferma che se abbiamo una prova della reductio ad absurdum del principio del terzo escluso, possiamo dedurre il principio del terzo escluso stesso.

Abbiamo così dimostrato che i due principi logici sono equivalenti.

4.3 Logica del primo ordine

Possiamo estendere il nostro sistema logico per includere quantificatori del primo ordine, come il quantificatore universale $\forall$ e il quantificatore esistenziale $\exists$. Ricordiamo che quello che fa il quantificatore universale è affermare che una certa proprietà vale per tutti gli elementi di un certo insieme. In Agda, possiamo definire il quantificatore universale $\forall$ come segue:

```
All : (A : Set) → (A → Prop) → Prop
All A P = (x : A) → P x
```

Qui abbiamo definito una funzione `All` che prende come argomenti un insieme A e una proprietà P che mappa gli elementi di A a proposizioni. La funzione restituisce una proposizione che afferma che per ogni elemento x in A , la proprietà $P(x)$ è vera. Agda ci consente di fare una cosa utile: possiamo cambiare la sintassi per rendere il quantificatore universale più leggibile:

```
syntax All A (λ x → P) = ∀[ x ∈ A ] P
```

Invece di scrivere `All A (λ x → P)`, possiamo ora scrivere `∀[ x ∈ A ] P`, che è più simile alla notazione matematica tradizionale.

```
-- Definiamo alcune variabili (ovvero proprietà) per il nostro esempio
variable PP QQ : A → Prop
-- Proviamo a dimostrare che la congiunzione di due proprietà universali è equivalente
-- alla proprietà universale della congiunzione delle due proprietà.
all-and : ∀[ x ∈ A ] (PP x ∧ QQ x) ⇔
  (∀[ x ∈ A ] (PP x)) ∧ (∀[ x ∈ A ] (QQ x))
proj1 all-and f = (λ x → proj1 (f x)) , (λ x → proj2 (f x))
proj2 all-and (f , g) = λ x → (f x) , (g x)
```

Quindi se un elemento x appartiene ad un insieme A e soddisfa sia la proprietà PP che la proprietà QQ , allora possiamo concludere che x soddisfa entrambe le proprietà.

- La funzione `all-and` prende come argomento una prova che per ogni elemento x in A , la congiunzione $PP(x) \wedge QQ(x)$ è vera, e restituisce una prova che la congiunzione delle proprietà universali $(\forall[x \in A](PPx)) \wedge (\forall[x \in A](QQx))$ è vera.
- Nel corpo della funzione, usiamo il pattern matching per distinguere i due casi:
 - Per la prima direzione, usiamo il campo `proj1` per estrarre la prova di $PP(x)$ e il campo `proj2` per estrarre la prova di $QQ(x)$ dalla congiunzione $PP(x) \wedge QQ(x)$.
 - Per la seconda direzione, usiamo le prove f e g per costruire la congiunzione $PP(x) \wedge QQ(x)$.

Per quanto riguarda il quantificatore esistenziale $\exists$, è leggermente più complicato da definire in Agda, poiché dobbiamo rappresentare sia l'elemento che soddisfa la proprietà che la prova che

l'elemento soddisfa la proprietà. Prima di tutto, dobbiamo creare un tipo di record per rappresentare il quantificatore esistenziale:

```
record  $\Sigma$  (A : Set) (P : A  $\rightarrow$  Prop) : Set where
  constructor _,_
  field
    witness : A
    proof   : P witness
```

Qui abbiamo definito un record Σ che prende come argomenti un insieme A e una proprietà P che mappa gli elementi di A a proposizioni. Il record ha due campi:

- `witness` rappresenta l'elemento di A che soddisfa la proprietà P .
- `proof` rappresenta la prova che l'elemento `witness` soddisfa la proprietà P .

Ora possiamo operare analogamente alla funzione `All` per definire il quantificatore esistenziale $\exists$:

```
Ex : (A : Set)  $\rightarrow$  (A  $\rightarrow$  Prop)  $\rightarrow$  Prop
Ex A P =  $\Sigma$  A P

syntax Ex A ( $\lambda$  x  $\rightarrow$  P) =  $\exists$ [ x  $\in$  A ] P
```

Banalmente abbiamo definito una funzione `Ex` che prende come argomenti un insieme A e una proprietà P che mappa gli elementi di A a proposizioni. La funzione restituisce una proposizione che afferma che esiste un elemento x in A tale che la proprietà $P(x)$ è vera. Come prima, abbiamo anche definito una sintassi più leggibile per il quantificatore esistenziale, simile alla notazione matematica tradizionale. Possiamo provare ora una proprietà interessante riguardante il quantificatore esistenziale e la disgiunzione:

```
ex-or :  $\exists$ [ x  $\in$  A ] (PP x  $\vee$  QQ x)  $\Leftrightarrow$  ( $\exists$ [ x  $\in$  A ] PP x)  $\vee$  ( $\exists$ [ x  $\in$  A ] QQ x)
proj1 ex-or (a, inj1 pa) = inj1 (a , pa)
proj1 ex-or (a, inj2 qa) = inj2 (a , qa)
proj2 ex-or (inj1 (a , pa)) = a , inj1 pa
proj2 ex-or (inj2 (a , qa)) = a , inj2 qa
```

Cerchiamo di capire questa definizione:

- La funzione `ex-or` prende come argomento una prova che esiste un elemento x in A tale che la disgiunzione $PP(x) \vee QQ(x)$ è vera, e restituisce una prova che la disgiunzione delle proprietà esistenziali $(\exists[x \in A]PPx) \vee (\exists[x \in A]QQx)$ è vera.
- Nel corpo della funzione, usiamo il pattern matching per distinguere i due casi:
 - Per la prima direzione, se la prova è costruita usando `inj1`, allora estraiamo l'elemento a e la prova pa di $PP(a)$, e costruiamo la disgiunzione usando `inj1`.

- Se la prova è costruita usando inj_2 , allora estraiamo l'elemento a e la prova qa di $QQ(a)$, e costruiamo la disgiunzione usando inj_2 .
- Per la seconda direzione, se la disgiunzione è costruita usando inj_1 , allora estraiamo l'elemento a e la prova pa di $PP(a)$, e costruiamo la coppia $(a, \text{inj}_1 pa)$.
- Se la disgiunzione è costruita usando inj_2 , allora estraiamo l'elemento a e la prova qa di $QQ(a)$, e costruiamo la coppia $(a, \text{inj}_2 qa)$.

4.3.1 Assioma della scelta

Come ultimo esempio, vediamo come possiamo modellare l'assioma della scelta in Agda. L'assioma della scelta è uno dei principi più discussi della teoria degli insiemi e afferma che dato un insieme di insiemi non vuoti, è possibile scegliere un elemento da ciascun insieme per formare un nuovo insieme. Questo assioma ci dice soltanto che **esiste** una tale funzione di scelta, senza fornirci un modo esplicito per costruirla. In Agda possiamo modellare l'assioma della scelta come segue:

```
-- Definiamo RR come una relazione tra gli insiemi A e B
variable RR : A → B → Prop

-- L'assioma della scelta dice
-- se per ogni x in A esiste un y in B tale che R x y
-- allora esiste una funzione f da A a B tale che per ogni x in A R x (f x)
ac : (∀[ x ∈ A ] ∃[ y ∈ B ] RR x y) ⇒
    (∃[ f ∈ (A → B) ] ∀[ x ∈ A ] RR x (f x))
ac h = (λ x → Σ.witness (h x)) , (λ x → Σ.proof (h x))
```

Seguiamo il ragionamento dietro questa definizione:

- La funzione ac prende come argomento una prova che per ogni elemento x in A , esiste un elemento y in B tale che la relazione $RR(x, y)$ è vera.
- La funzione restituisce una prova che esiste una funzione f da A a B tale che per ogni elemento x in A , la relazione $RR(x, f(x))$ è vera.
- Nel corpo della funzione, costruiamo la funzione f come una funzione anonima $\lambda x \rightarrow \Sigma.witness (h x)$, che prende un elemento x in A e restituisce l'elemento y in B scelto dalla prova $h(x)$.
- Costruiamo anche la prova che per ogni elemento x in A , la relazione $RR(x, f(x))$ è vera come una funzione anonima $\lambda x \rightarrow \Sigma.proof (h x)$, che prende un elemento x in A e restituisce la prova che la relazione $RR(x, f(x))$ è vera dalla prova $h(x)$.
- Quindi, questa definizione afferma che se abbiamo una prova che per ogni elemento in A esiste un elemento in B che soddisfa la relazione RR , allora possiamo costruire una funzione di scelta f che sceglie un elemento da B per ogni elemento in A in modo che la relazione RR sia soddisfatta.

5 Metalogica in Agda

Una cosa interessante di Agda è che possiamo anche modellare concetti di metalogica (quindi possiamo usare Agda per ragionare su sistemi logici stessi), grazie alla sua natura di linguaggio di programmazione e sistema di dimostrazione interattivo. Ad esempio, possiamo definire un semplice sistema logico proposizionale in Agda. Proviamo a modellare alcuni concetti della **logica minimale**, che è una logica proposizionale più debole della logica classica poiché contiene solo l'implicazione e la negazione come connettivi logici. Quest'ultima rifiuta il principio del terzo escluso e l'ex falso quodlibet. Andremo inoltre a definire la deduzione naturale come sistema deduttivo.

5.1 Definizione delle proposizioni e formule

In Agda, possiamo definire un tipo per rappresentare le proposizioni e le formule logiche.

```
open import Data.String

data Form : Set where
  atom : String → Form
  _[⇒]_ : Form → Form → Form
```

Questo non sarebbe un modo molto efficiente per rappresentare le formule logiche, ma per scopi didattici va più che bene. Quindi abbiamo definito un tipo `Form` con due costruttori:

- `atom` rappresenta una proposizione atomica, identificata da una stringa e mi permette di creare proposizioni come "P", "Q", "R", ecc.
- `_ [⇒] _` rappresenta l'implicazione tra due formule logiche.

5.2 Definizione dei contesti

In logica, un contesto è un insieme di assunzioni o ipotesi da cui possiamo dedurre altre proposizioni. In Agda, possiamo rappresentare un contesto come una lista di formule logiche. Anche in questo caso, tocca definirlo in maniera ricorsiva.

```
data Context : Set where
  • : Con
  _▷_ : Con → Form → Con
```

Ricordiamo anche di dare la precedenza all'operatore `▷` e `⇒`:

```
infixr 8 _▷_
infixr 9 _[⇒]_
```

Per evitare di definire ogni volta le variabili e contesti con dei nomi ad ogni definizione (possiamo usare le variabili implicite di Agda):

```
variable Γ Δ : Con
variable ψ φ : Form
```

5.3 Definizione delle regole di deduzione naturale

Dobbiamo ora definire $\vdash$ ovvero il giudizio di deducibilità. Ora possiamo definire le regole di deduzione naturale per la logica minimale (le linee orizzontali chiaramente separano le ipotesi dalle conclusioni, ma non sono codificate all'interno di Agda).

```
infix 5 _⊢_
data _⊢_ : Con → Form → Set where
  -----
  zero : Γ ▷ φ ⊢ φ

  suc : Γ ⊢ ψ →
    -----
    Γ ▷ φ ⊢ ψ

  lam : Γ ▷ φ ⊢ ψ → -- introduzione dell'implicazione
    -----
    Γ ⊢ φ [⇒] ψ

  app : Γ ⊢ φ [⇒] ψ → Γ ⊢ φ → -- modus ponens
    -----
    Γ ⊢ ψ
```

Questa definizione è più complessa perché possiede diversi costruttori:

- **zero**: rappresenta la regola di assunzione, che afferma che se una formula ϕ è nel contesto Γ , allora possiamo dedurre ϕ da Γ .
- **suc**: rappresenta la regola di indebolimento, che afferma che se possiamo dedurre ψ da Γ , allora possiamo anche dedurre ψ da Γ esteso con una nuova formula ϕ .
- **lam**: rappresenta la regola di introduzione dell'implicazione, che afferma che se possiamo dedurre ψ da Γ esteso con ϕ , allora possiamo dedurre l'implicazione $\phi \Rightarrow \psi$ da Γ .

- app: rappresenta la regola di eliminazione dell'implicazione (modus ponens), che afferma che se possiamo dedurre $\phi \Rightarrow \psi$ da Γ e possiamo dedurre ϕ da Γ , allora possiamo dedurre ψ da Γ .

$$\frac{}{\Gamma, \varphi \vdash \varphi} \text{ (zero)}$$

$$\frac{\Gamma \vdash \psi}{\Gamma, \varphi \vdash \psi} \text{ (suc)}$$

$$\frac{\Gamma, \varphi \vdash \psi}{\Gamma \vdash \varphi \Rightarrow \psi} \text{ (lam)}$$

$$\frac{\Gamma \vdash \varphi \Rightarrow \psi \quad \Gamma \vdash \varphi}{\Gamma \vdash \psi} \text{ (app)}$$

5.4 Esempio di prova in Agda

Proviamo a costruire un esempio semplice in Agda usando le regole che abbiamo appena creato. Se io volessi dimostrare che da P implica (banalmente) P e quindi costruire una prova di $P \Rightarrow P$, potrei farlo così:

```
I : • ⊢ (atom "P") [⇒] (atom "P")
-- λ x → x
I = lam zero
```

Sto cercando di provare che dal contesto vuoto (rappresentato da $\bullet$), posso dedurre che P implica P (rappresentato da $(\text{atom "P"}) [\Rightarrow] (\text{atom "P"})$). Sto usando la regola di introduzione dell'implicazione (lam) per costruire questa prova applicando l'assioma che ci permette di "chiudere" la derivazione. Quindi una visualizzazione grafica tramite deduzione naturale sarebbe:

$$\frac{[P]^1}{P \Rightarrow P} (\rightarrow I)^1$$

🔗 Osservazione 5.1

Per semplicità definiamo come assunzione il nostro costruttore zero, mentre suc non fa nulla di più che estendere il contesto con una nuova assunzione che però non viene usata nella dimostrazione (quindi ci permette di creare nuove variabili). Il costruttore lam rappresenta l'introduzione dell'implicazione e app rappresenta l'eliminazione dell'implicazione (modus ponens).

Cerchiamo ora invece di dimostrare:

$$\vdash (P \rightarrow Q \rightarrow R) \rightarrow (Q \rightarrow P \rightarrow R)$$

In Agda, possiamo costruire questa prova come segue:

```
sw : • ⊢ ((atom "P") [⇒] (atom "Q")) [⇒] (atom "R")
[⇒] ((atom "Q") [⇒] (atom "P")) [⇒] (atom "R")
sw = lam (lam (lam (app (app (suc (suc zero)) zero) (suc zero))))
```

Questa definizione può sembrare complessa a prima vista, ma notiamo che stiamo costruendo la prova passo dopo passo utilizzando le regole di deduzione naturale che abbiamo definito. La forma equivalente in deduzione naturale sarebbe:

$$\frac{\frac{\frac{[P \rightarrow (Q \rightarrow R)]^1 \quad [P]^3}{Q \rightarrow R} (\rightarrow E) \quad [Q]^2}{R} (\rightarrow E) \quad \frac{R}{P \rightarrow R} (\rightarrow I)^3}{Q \rightarrow (P \rightarrow R)} (\rightarrow I)^2}{(P \rightarrow Q \rightarrow R) \rightarrow (Q \rightarrow P \rightarrow R)} (\rightarrow I)^1$$

💡 Osservazione 5.2

Notiamo come abbiamo applicato la regola di introduzione dell'implicazione (1am) tre volte e la regola di eliminazione dell'implicazione (app) due volte, proprio come nella derivazione in Agda.

Ora che abbiamo visto la deduzione naturale in maniera grafica, proviamo a srotolare quello che abbiamo fatto invece in agda. Il modo intuitivo di costruire la prova in Agda e provarla a guardarla con un approccio bottom-up (chi ha esperienza con questo tipo di esercizi, sa che è il modo più semplice per affrontarli):

- La tesi è di tipo $(P \rightarrow Q \rightarrow R) \rightarrow (Q \rightarrow P \rightarrow R)$, quindi per costruire una prova di questo tipo, dobbiamo sicuramente introdurre un'ipotesi $P \rightarrow Q \rightarrow R$ nelle assunzioni. Usiamo quindi 1am per introdurre questa ipotesi.
- Ora, la tesi diventa $Q \rightarrow P \rightarrow R$, quindi dobbiamo introdurre un'altra ipotesi Q nelle assunzioni. Usiamo di nuovo 1am per introdurre questa ipotesi.
- Ora, la tesi diventa $P \rightarrow R$, quindi dobbiamo introdurre un'ultima ipotesi P nelle assunzioni. Usiamo ancora 1am per introdurre questa ipotesi.
- A questo punto abbiamo applicato 3 volte 1am, e la tesi diventa R .
- Possiamo ottenere R applicando Modus Ponens a $Q \rightarrow R$ e Q .
- A sua volta $Q \rightarrow R$ può essere ottenuto applicando Modus Ponens a $P \rightarrow Q \rightarrow R$ e P .
- Quindi alla fine le assunzioni che abbiamo chiuso sono proprio:
 - $P \rightarrow Q \rightarrow R$ (chiusa con la prima 1am),
 - Q (chiusa con la seconda 1am),
 - P (chiusa con la terza 1am).

Prendiamo ora come esempio la proposizione:

$$(P \rightarrow Q) \rightarrow (Q \rightarrow P)$$

Se provassimo a dimostrare questa proposizione in Agda, andremo ad incontrare un problema:

```
bad : • ⊢ (atom "P" [⇒] atom "Q") [⇒] (atom "Q" [⇒] atom "P")
```

Questa proposizione è falsa, infatti per $P = \text{False}$ e $Q = \text{True}$, non vale. Quindi non esiste una prova di questa proposizione.

5.5 Semantica delle formule

Ora che abbiamo visto la sintassi possiamo anche definire una semantica per le formule logiche. Proviamo ora a definire una interpretazione della semantica in Agda. Definiremo **Env** che sarà la nostra "valutazione" che quindi data una formula atomica ci ritorna una proposizione. Nella semantica classica scriveremmo:

$$\rho : \text{Atoms} \rightarrow \{\text{True}, \text{False}\}$$

Poi definiremo la funzione di valutazione delle formule logiche dove $\llbracket \phi \rrbracket_\rho$ rappresenta la valutazione della formula ϕ sotto l'interpretazione ρ .

🔗 Osservazione 5.3

Per ogni formula ϕ e ogni interpretazione ρ , possiamo calcolare la proposizione $\llbracket \phi \rrbracket_\rho$ che rappresenta "il significato di ϕ sotto ρ ".

```
Env = String → Prop
```

```
[[_]] : Form → Env → Prop
```

```
[[ atom x ]] ρ = ρ x
```

```
[[ φ [⇒] ψ ]] ρ = [[ φ ]] ρ → [[ ψ ]] ρ
```

Cerchiamo di scomporre questa definizione:

- $\llbracket \text{atom } x \rrbracket \rho = \rho x$ significa che la valutazione di una formula atomica è semplicemente il valore assegnato a quella formula dall'interpretazione ρ .
- $\llbracket \phi [⇒] \psi \rrbracket \rho = \llbracket \phi \rrbracket \rho \rightarrow \llbracket \psi \rrbracket \rho$ significa che la valutazione di un'implicazione è una funzione che prende la valutazione della formula ϕ e restituisce la valutazione della formula ψ .

Infatti un esempio di environment potrebbe essere:

```
env : Env
```

```
env "P" = ⊥
```

```
env "Q" = ⊤
```

```
env _ = ⊥
```

Definiamo ora lo stato di valutazione per i contesti:

```

[[_]]* : Con → Env → Prop
[[ • ]]* ρ = ⊤
[[ Γ ▷ ϕ ]]* ρ = [[ Γ ]]* ρ × [[ ϕ ]] ρ

```

In questa definizione:

- $[[•]]^* \rho = \top$ significa che la valutazione del contesto vuoto è sempre vera.
- $[[\Gamma \triangleright \phi]]^* \rho = [[\Gamma]]^* \rho \times [[\phi]] \rho$ significa che la valutazione di un contesto esteso con una formula è la congiunzione della valutazione del contesto originale e della valutazione della nuova formula.

5.6 Teorema di correttezza

Ora possiamo affermare e dimostrare il teorema di correttezza per il nostro sistema logico minimale in Agda. Il teorema di correttezza afferma che se una formula ψ è deducibile da un contesto Γ (cioè $\Gamma \vdash \psi$), allora la valutazione di ψ sotto qualsiasi interpretazione ρ che soddisfa il contesto Γ è vera. In poche parole, se possiamo dimostrare una formula a partire da un insieme di assunzioni, allora quella formula è semanticamente valida in ogni interpretazione che rende vere le assunzioni. Questo teorema è molto forte perché collega la sintassi (le regole di deduzione) con la semantica (la valutazione delle formule).

$$\Gamma \vdash \psi \iff \Gamma \models \psi$$

Quindi in poche parole: "Ogni giudizio derivabile è semanticamente valido". È uno strumento fondamentale per garantire che il nostro sistema logico sia **coerente** e **affidabile**. Un sistema che non dispone di un teorema di correttezza potrebbe permettere di derivare proposizioni che non sono semanticamente valide, portando a contraddizioni e risultati errati. Formalizzare e dimostrare questo teorema in Agda ci permette di avere una prova meccanica della correttezza del nostro sistema logico. Sono passaggi abbastanza complessi quindi affrontiamoli step by step.

```

sound : Γ ⊢ ψ → [[ Γ ]]* ρ → [[ ψ ]] ρ
sound zero ( _ , p ) = p
sound (suc d) (γ , _) = sound d γ
sound (lam d) γ p = sound d (γ , p)
sound (app d e) γ = sound d γ (sound e γ)

```

In questa definizione:

- Abbiamo definito una funzione `sound` che prende una prova di deducibilità $\Gamma \vdash \psi$ e una valutazione del contesto $[[\Gamma]]^* \rho$, e restituisce la valutazione della formula $[[\psi]] \rho$.
- Ora, facciamo pattern matching sui costruttori della prova di deducibilità.

- Nel caso di `zero`, estraiamo la valutazione della formula ψ direttamente dalla valutazione del contesto. Quindi se abbiamo una prova che ϕ è nel contesto Γ , allora possiamo ottenere la valutazione di ϕ dalla valutazione del contesto.
- Nel caso di `suc`, ignoriamo la nuova formula aggiunta al contesto e continuiamo a valutare la prova originale. Quindi se possiamo dedurre ψ da Γ , allora possiamo anche dedurlo da Γ esteso con una nuova formula ϕ .
- Nel caso di `lam`, estendiamo la valutazione del contesto con la valutazione della nuova formula ϕ e continuiamo a valutare la prova originale. Quindi se possiamo dedurre ψ da Γ esteso con ϕ , allora possiamo dedurre l'implicazione $\phi \Rightarrow \psi$ da Γ . Ricordiamo che `lam` come costruttore rappresenta l'introduzione dell'implicazione che richiede un'ipotesi del tipo $\Gamma, \phi \vdash \psi$.
- Nel caso di `app`, applichiamo la valutazione della prova dell'implicazione alla valutazione della prova della premessa. Quindi se possiamo dedurre $\phi \Rightarrow \psi$ da Γ e possiamo dedurre ϕ da Γ , allora possiamo dedurre ψ da Γ . Come possiamo notare infatti `app` rappresenta il *modus ponens* che richiede due prove: una per l'implicazione e una per la premessa.

Ora che abbiamo costruito la funzione `sound`, possiamo affermare che il nostro sistema logico minimale è corretto. Ricordiamoci della nostra funzione `bad` che non riuscivamo a dimostrare perché appunto la proposizione non era valida. Ora possiamo usare il teorema di correttezza per spiegare perché non esiste una prova per quella proposizione. A differenza di prima, ora `bad` non è una prova di quella proposizione, ma una *prova della negazione* di quella proposizione.

```
bad : ¬ (• ⊢ (atom "P" [⇒] atom "Q") [⇒] (atom "Q" [⇒] atom "P"))
bad d = sound {ρ = env} d tt (λ _ → tt) tt

-- where env was defined as:
-- env : Env
-- env "P" = ⊥
-- env "Q" = ⊤
```

In questa definizione:

- Abbiamo definito una funzione `bad` che prende una prova di deducibilità $\bullet \vdash (P \Rightarrow Q) \Rightarrow (Q \Rightarrow P)$ e restituisce una prova della negazione di quella proposizione. (Ricordiamo che $\neg A$ è definito come $A \rightarrow \perp$, quindi una prova di $\neg A$ è una funzione che prende una prova di A e restituisce una contraddizione). Mostra che se esistesse una prova di $\bullet \vdash (P \Rightarrow Q) \Rightarrow (Q \Rightarrow P)$, allora potremmo ottenere una contraddizione.
- Usiamo la funzione `sound` per ottenere la valutazione della proposizione sotto l'interpretazione `env`, che in questo caso abbiamo deciso di essere quella che assegna P a falso e Q a vero. Questo non è nient'altro che il **contromodello** che mostra che la proposizione non è valida per questa interpretazione specifica.

- Passiamo la valutazione del contesto vuoto tt (che è sempre vera, poiché il contesto vuoto lo abbiamo definito come vero) e le valutazioni delle formule $P \Rightarrow Q$ e $Q \Rightarrow P$ sotto l'interpretazione env .
- L'argomento $(\lambda _ \rightarrow tt) \ tt$ dipendono dal tipo di interpretazione. Essendo che la nostra `sound` richiama se stessa ricorsivamente nelle sottoprove che compongono la prova, dovresti aver bisogno di aggiungere degli argomenti aggiuntivi "dummy" per le assunzioni. Essenzialmente, sono le "prove delle premesse" che sono necessarie per usare `sound`, ma essendo che siamo in un contesto vuoto, queste prove sono banali (cioè vere).
- Espandiamo: quello che `sound` deve tornare è una prova di $\llbracket \psi \rrbracket \rho$ dove:

$$\llbracket (atom \ "P") [\Rightarrow] (atom \ "Q") \rrbracket env = \llbracket atom \ "P" \rrbracket env \rightarrow \llbracket atom \ "Q" \rrbracket env = \perp \rightarrow \top = \top$$

$$\llbracket (atom \ "Q") [\Rightarrow] (atom \ "P") \rrbracket env = \llbracket atom \ "Q" \rrbracket env \rightarrow \llbracket atom \ "P" \rrbracket env = \top \rightarrow \perp = \perp$$

Quindi il tutto diventa:

$$\top \rightarrow \perp$$

Che quindi è falso, inabitato. Quindi `sound` d `tt` forza Agda a produrre una prova di $\perp$, che è impossibile e quindi genera una contraddizione.

- Quindi, non esiste una prova di $\bullet \vdash (P \Rightarrow Q) \Rightarrow (Q \Rightarrow P)$, il che dimostra che la proposizione non è valida.

5.7 Teorema di completezza (e perché non è sempre realizzabile in Agda)

Il teorema di completezza afferma che se una formula ψ è semanticamente valida in ogni interpretazione che soddisfa un contesto Γ (allora $\Gamma \models \psi$), allora esiste una prova di deducibilità $\Gamma \vdash \psi$. Questo strumento è anch'esso fondamentale come la `soundness`, ma chiaramente molto più forte (e più complesso da dimostrare) poiché ci permette di costruire prove a partire da validità semantiche. Quello che dovremmo fare in Agda sarebbe definire una funzione che prende una valutazione semantica e restituisce una prova di deducibilità.

```
complete : {ρ : Env} → (∀ ρ →  $\llbracket \Gamma \rrbracket * \rho \rightarrow \llbracket \phi \rrbracket \rho$ ) →  $\Gamma \vdash \phi$ 
compl = ?
```

Quello che stiamo cercando di dire è: se ψ è vero in ogni interpretazione che rende vere le formule in Γ , allora possiamo costruire una prova sintattica di ψ a partire da Γ . Tuttavia **non** è costruttivamente realizzabile in Agda. O almeno, non in modo semplice per ogni tipo di logica.

- Agda è basata sulla logica costruttiva che è più debole della logica classica e non supporta alcuni principi logici che sono necessari per dimostrare la completezza in generale come il principio del terzo escluso.

- La funzione `compl` non è computabile in generale: non esiste un algoritmo che, dato un'interpretazione semantica, possa sempre costruire una prova sintattica.

Tuttavia è comunque possibile farlo per **certe** logiche specifiche che solitamente sono ristrette o decidibili. È stato provato e formalizzato in Agda per la logica proposizionale classica [LA15] e per altre logiche.

Riferimenti bibliografici

- [LA15] Ambrus Kaposi Leran Cai e Thorsten Altenkirch. «Formalising the Completeness Theorem of Classical Propositional Logic in Agda (Proof Pearl)». In: (2015), pp. 1–16.